Importance and Rejection Sampling

Giulio Beltramin

2026-10-12

Importance Sampling

1. Usually employed to approximate expected values w.r.t. a random variable θ on Θ

$$\mathbb{E}_{\theta \sim p}[h(\theta)]$$

   when p is intractable. E.g., $p(\theta)$ is the posterior distribution of a model

2. Popular in Reinforcement Learning: e.g. "Importance Sampling Techniques for Policy Optimization" by Metelli et al (2020) Journal of Machine Learning Research

3. Can be used also to approximate the intractable distribution $p(\cdot)$

Example 1 - Standard Gaussian distribution

Let

$$p(\theta) = \frac{1}{\sqrt{2\pi}} \mathrm{e}^{-\theta^2/2} \quad \text{and} \quad q(\theta) = \frac{1}{\sqrt{2\pi}\sigma} \mathrm{e}^{-\theta^2/(2\sigma^2)}$$

Let's compare the performance in estimating $\mathbb{E}_{\theta \sim p}[\theta]$.

```
# First, consider standard Monte Carlo
nrep <- 1000
nsamples <- 100

# Draw standard normal variates (matrix: 1000 rows, 100 columns)
thetas <- matrix(rnorm(nrep * nsamples), nrow = nrep, ncol = nsamples)

# Compute cumulative Monte Carlo averages across rows (samples 1..100)
# apply with margin = 1 operates row-wise; t() transposes back to 1000 X 100
mc_avg <- t(apply(thetas, 1, function(row) cumsum(row) / 1:nsamples))

# Compute mean and standard deviation across iterations (columns)
mean_vec <- colMeans(mc_avg)
sd_vec <- apply(mc_avg, 2, sd)

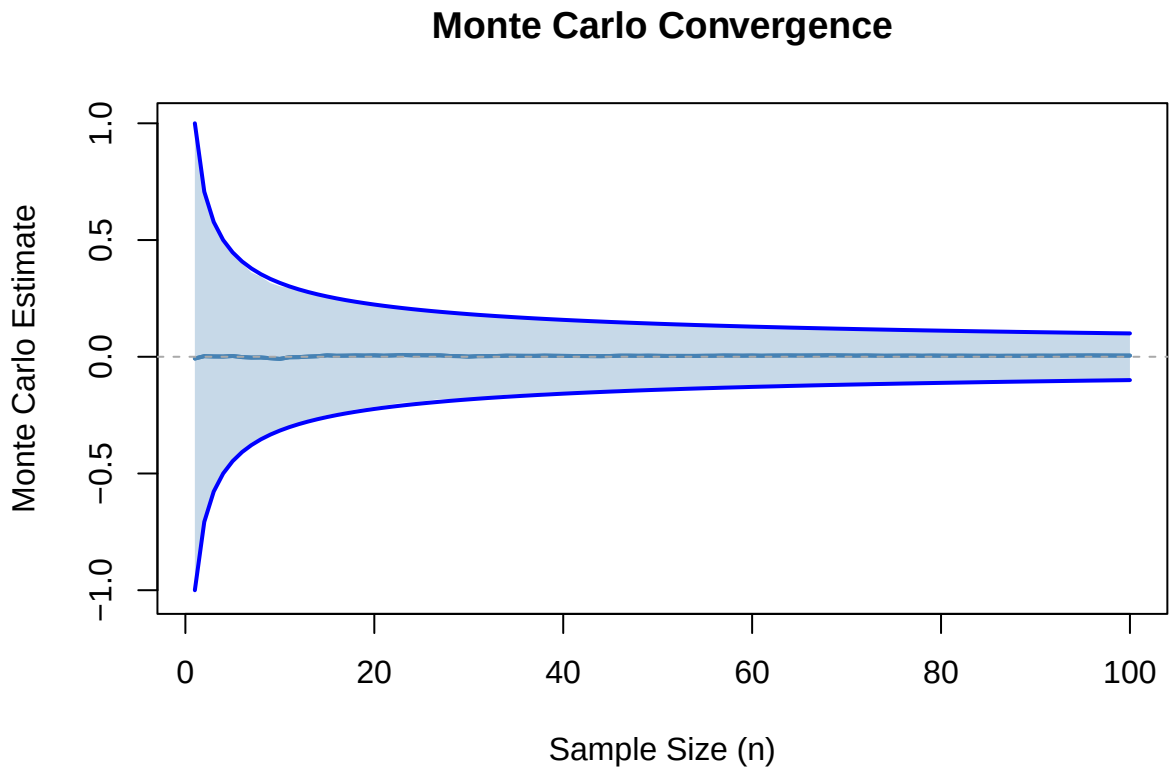
n_grid <- 1:nsamples

# Base plot line
plot(
  n_grid, mean_vec, type = "l", col = "steelblue", lwd = 2,
  ylim = range(c(mean_vec - sd_vec, mean_vec + sd_vec)),
```

```
  xlab = "Sample Size (n)", ylab = "Monte Carlo Estimate",
  main = "Monte Carlo Convergence"
)

# Add shaded region
polygon(
  c(n_grid, rev(n_grid)),
  c(mean_vec - sd_vec, rev(mean_vec + sd_vec)),
  col = rgb(70/255, 130/255, 180/255, 0.3),
  border = NA
)

rate <- 1/sqrt(n_grid)
lines(n_grid, mean_vec, col = "steelblue", lwd = 2)
lines(n_grid, rate, col = "blue", lwd = 2)
lines(n_grid, -rate, col = "blue", lwd = 2)
abline(h = 0, lty = 2, col = "darkgray")
```




```
set.seed(123)
nrep <- 1000
nsamples <- 100
sigmas <- c(0.2, 0.4, 0.8, 1.5)

# Setup 1 X 4 panel layout
par(mfrow = c(1, length(sigmas)), mar = c(4, 4, 3, 1))
```

```r
for (sig in sigmas) {
  # Sample from proposal
  thetas <- matrix(rnorm(nrep * nsamples, mean = 0, sd = sig), nrow = nrep, ncol = nsamples)

  # Calculate importance weights and weighted samples
  weights <- dnorm(thetas) / dnorm(thetas, mean = 0, sd = sig)
  weighted_thetas <- thetas * weights

  # Calculate cumulative averages row-wise
  is_avg <- t(apply(weighted_thetas, 1, function(row) cumsum(row) / 1:nsamples))

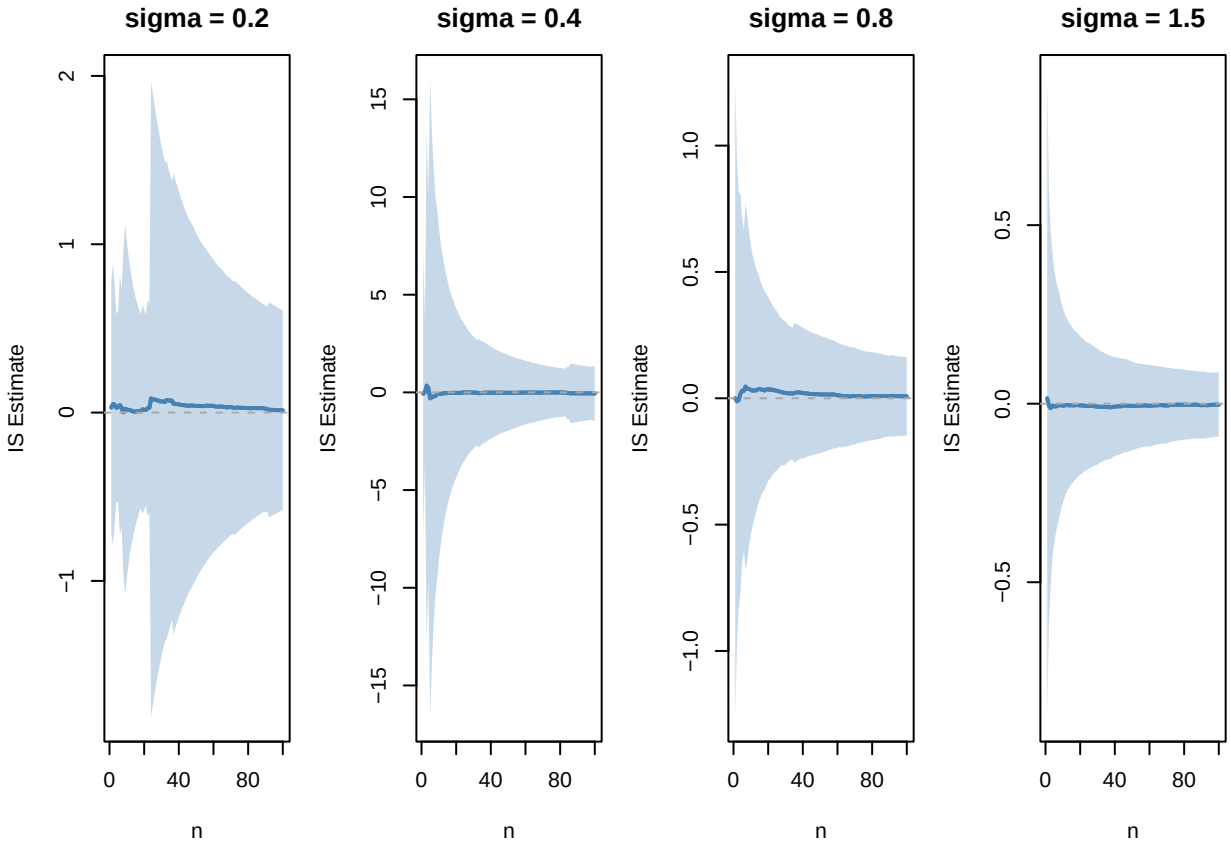
  # Column-wise mean and standard deviation
  mean_vec <- colMeans(is_avg)
  sd_vec <- apply(is_avg, 2, sd)
  n_grid <- 1:nsamples

  plot(
    n_grid, mean_vec, type = "l", col = "steelblue", lwd = 2,
    ylim = range(c(mean_vec - sd_vec, mean_vec + sd_vec)),
    xlab = "n", ylab = "IS Estimate", main = paste0("sigma = ", sig)
  )

  polygon(
    c(n_grid, rev(n_grid)),
    c(mean_vec - sd_vec, rev(mean_vec + sd_vec)),
    col = rgb(70/255, 130/255, 180/255, 0.3),
    border = NA
  )

  lines(n_grid, mean_vec, col = "steelblue", lwd = 2)

  abline(h = 0, lty = 2, col = "darkgray")
}
```




```
# Reset graphics panel
par(mfrow = c(1, 1))
```

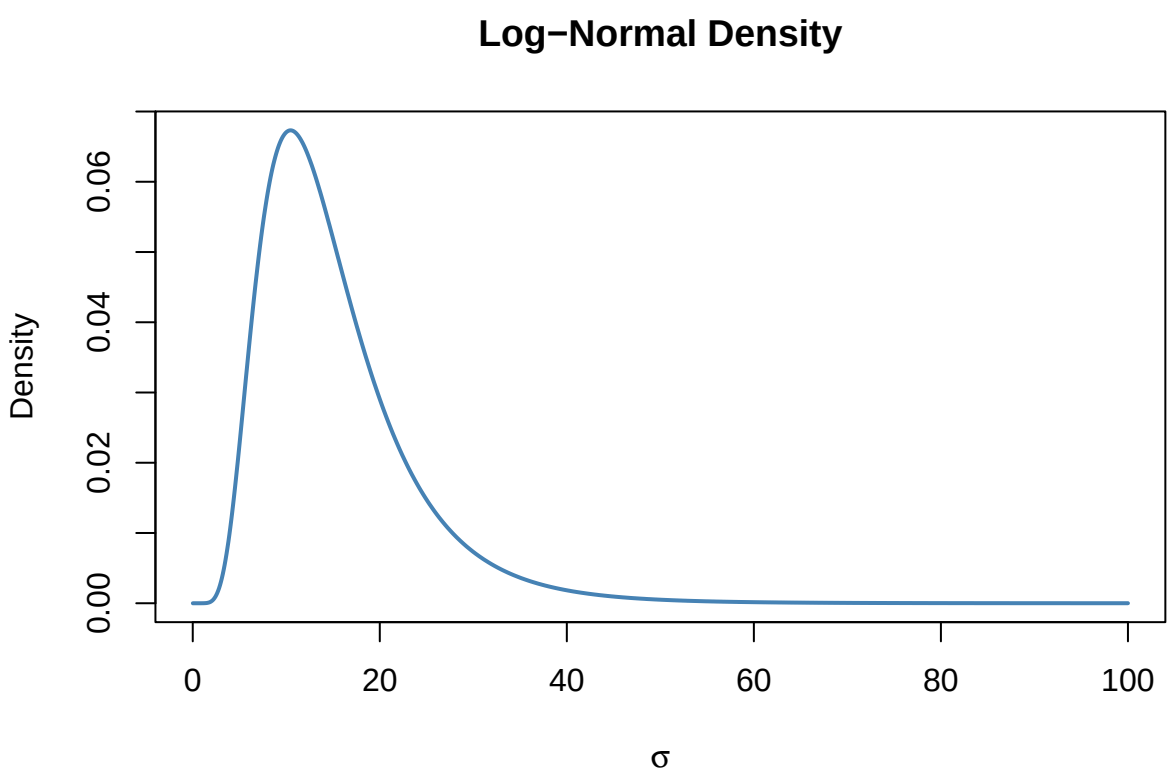
Homework: find the optimal σ (it's not 1!) *Hint*: you have to minimize the variance of the IS estimator.

Example 2- Importance Sampling for posterior inference on the `VetBP` dataset

$$
\begin{aligned}
y_1, \ldots, y_n \mid \mu, \sigma &\overset{iid}{\sim} \mathcal{N}(\mu, \sigma^2) \\
\mu &\sim \mathcal{N}(124, 7.5^2) \\
\sigma &\sim \text{logN}(\log(13.43), 0.5)
\end{aligned}
$$

```
sigrange <- seq(0.01, 100, length.out = 1000)
density_vals <- dlnorm(sigrange, meanlog = log(13.43), sdlog = 0.5)

plot(
  sigrange, density_vals, type = "l", col = "steelblue", lwd = 2,
  xlab = expression(sigma), ylab = "Density", main = "Log-Normal Density"
)
```




```
df <- read.csv("vetbp.csv")
sbp <- df$SBP_00
```

The choice of q

We are interested in computing expectations w.r.t. $\pi(\mu, \sigma \mid \mathbf{y})$, so q must be a probability on $\mathbb{R} \times [0, +\infty)$

For instance, we could consider q to be the prior!

Let's start with a simple computation of the posterior expected value

$$\mathbb{E}[\sigma \mid \mathbf{y}]$$

```
# Prior sampling function: draws mu ~ Normal, sigma ~ LogNormal
sample_prior <- function(size, mu_mean, mu_std, sig_loc, sig_scale) {
  mu <- rnorm(size, mean = mu_mean, sd = mu_std)
  sigma <- rlnorm(size, meanlog = sig_loc, sdlog = sig_scale)

  # Matrix with columns [mu, sigma]
  cbind(mu = mu, sigma = sigma)
}

# Normalized Importance Sampling weights
normalized_is_weights_prior <- function(data, samples) {
  mu_samples <- samples[, 1]
  sigma_samples <- samples[, 2]
```

```
  log_lik <- numeric(nrow(samples))
  # For each sample w(theta) = L(y | theta) * pi(theta) / q(theta)
  # here pi(theta) = q(theta) because I am using the prior as proposal
  # so w(theta) = L(y | theta)
  for (i in 1:nrow(samples)) {
    log_lik[i] <- sum(dnorm(data, mean = mu_samples[i], sd = sigma_samples[i], log = TRUE))
  }

  log_weights <- (log_lik)

  # We should compute np.exp(weights) / np.sum(np.exp(weights)) --> leads to numerical errors (see also
  # Instead, use the "logsumexp" trick
  # https://en.wikipedia.org/wiki/LogSumExp

  max_log <- max(log_weights)
  lse <- max_log + log(sum(exp(log_weights - max_log)))

  # Compute normalized weights in probability space
  log_norm_weights <- log_weights - lse
  return(exp(log_norm_weights))
}
```

```
# Generate prior samples (10000 X 2 matrix: [mu, sigma])
samples <- sample_prior(10000, mu_mean = 124, mu_std = 7.5, sig_loc = log(13.43), sig_scale = 0.5)

# Calculate normalized importance weights
weights <- normalized_is_weights_prior(sbp, samples)
# Print sum of weights (should be 1.0)
cat("Sum of weights: ", sum(weights), "\n")
```

```
## Sum of weights:  1
```

```
# Calculate posterior mean of sigma (weighted average)
mean_sig <- sum(weights * samples[, 2])
cat("mean_sig: ", mean_sig, "\n")
```

```
## mean_sig:  15.57385
```

Is this a good proposal? We can look at the **effective sample size** (ESS), defined as:

$$ESS = \frac{1}{\sum w_i^2}$$

```
# Compute effective sample size
ess <-  1.0 / sum(weights ^ 2)
cat("ess: ", ess)
```

```
## ess:  57.5272
```

What if we use a better proposal? For instance, we know the mean really concentrates around the empirical average

$$q(\mu, \sigma) = \mathcal{N}(\mu; 134.5, 10) \times \mathcal{N}(\hat{\sigma}, 2)$$

where $\hat{\sigma} = \frac{1}{n}\sqrt{\sum(y_i - 134.5)^2}$

```r
# Proposal sampler: draws mu ~ Normal, sigma ~ Normal
sample_q <- function(size, mu_mean, mu_std, sig_loc, sig_scale) {
  mu    <- rnorm(size, mean = mu_mean, sd = mu_std)
  sigma <- rnorm(size, mean = sig_loc,  sd = sig_scale)

  cbind(mu = mu, sigma = sigma)
}

normalized_is_weights_q <- function(data, samples,
                                    prior_mu_mean, prior_mu_std,
                                    prior_sig_loc, prior_sig_scale,
                                    prop_mu_mean, prop_mu_std,
                                    prop_sig_loc, prop_sig_scale) {

  mu_samples    <- samples[, 1]
  sigma_samples <- samples[, 2]

  # For each sample w(theta) = L(y | theta) * pi(theta) / q(theta)
  # here pi(theta) != q(theta)
  log_liks <- apply(samples, 1, function(p) {
    if (p[2] <= 0) return(-Inf) # Guard against invalid sigma
    sum(dnorm(data, mean = p[1], sd = p[2], log = TRUE))
  })

  # mu ~ Normal(prior_mu_mean, prior_mu_std)
  # sigma ~ LogNormal(prior_sig_loc, prior_sig_scale)
  log_prior_mu  <- dnorm(mu_samples, mean = prior_mu_mean, sd = prior_mu_std, log = TRUE)
  log_prior_sig <- suppressWarnings(
    dlnorm(sigma_samples, meanlog = prior_sig_loc, sdlog = prior_sig_scale, log = TRUE)
  )
  log_prior_sig[sigma_samples <= 0] <- -Inf
  log_prior <- log_prior_mu + log_prior_sig

  # Compute log-proposal log q(mu, sigma) = q(mu)*q(sigma)
  # mu ~ Normal(prop_mu_mean, prop_mu_std)
  # sigma ~ Normal(prop_sig_loc, prop_sig_scale)
  log_prop_mu  <- dnorm(mu_samples, mean = prop_mu_mean, sd = prop_mu_std, log = TRUE)
  log_prop_sig <- dnorm(sigma_samples, mean = prop_sig_loc, sd = prop_sig_scale, log = TRUE)
  log_prop     <- log_prop_mu + log_prop_sig

  log_weights <- log_liks + log_prior - log_prop

  # LogSumExp trick for numerical normalization
  valid_idx <- is.finite(log_weights)
  if (!any(valid_idx)) {
    stop("All particle log-weights are non-finite. Check proposal overlap with prior/likelihood.")
  }

  max_log <- max(log_weights[valid_idx])
```

```
  lse <- max_log + log(sum(exp(log_weights[valid_idx] - max_log)))

  weights <- numeric(length(log_weights))
  weights[valid_idx] <- exp(log_weights[valid_idx] - lse)

  return(weights)
}
```

```
sigmahat = sqrt(mean((sbp - mean(sbp))^2))
sigmahat
```

```
## [1] 15.54465
```

```
samples <-  sample_q(10000, 134.5, sqrt(10), sigmahat, sqrt(2))
weights <-  normalized_is_weights_q(sbp, samples, 124, 7.5, log(13.43), 0.5,
                                    134.5, sqrt(10), sigmahat, sqrt(2))
cat("Sum of weights: ", sum(weights), "\n")
```

```
## Sum of weights:  1
```

```
mean_sig <- sum(weights * samples[, 1])
cat("mean_sig: ", mean_sig, "\n")
```

```
## mean_sig:  134.3814
```

```
ess <-  1.0 / sum(weights ^ 2)
cat("ess: ", ess, "\n")
```

```
## ess:  1835.692
```

Homework

Experiment a bit on the proposal distribution. For instance, we could use as proposal q the posterior from the Normal-Inverse Gamma model (which is conjugate and known in closed form).

Sampling from the posterior using Importance Sampling

Define

$$\hat{\pi}(\theta) = \sum_{j=1}^{M} \tilde{w}_j \delta_{\theta_j}$$

It can be shown that $\hat{\pi}(\theta) \to \pi(\theta \mid \mathbf{y})$ (using an appropriate metric/topology)

Therefore given $\hat{\pi}(\theta)$ we can obtain N samples $\vartheta_1, \ldots, \vartheta_N$ from (approximately) $\pi(\theta \mid \mathbf{y})$ as follows.

For $i = 1, \ldots, N$: 1. sample j from the discrete distribution on $\{1, \ldots, M\}$ with weights $\tilde{w}_1, \ldots, \tilde{w}_M$ 2. let $\vartheta_i = \theta_j$

Optimal proposal distribution

Given $h(x)$, the proposal distribution that minimizes the variance of the importance sampling estimator is

$$q^*(x) = \frac{|h(x)|p(x)}{\int |h(z)|p(z)\mathrm{d}z}$$

Proof:

The variance of the IS estimator is

$$\begin{aligned}\mathrm{Var}_{\theta\sim q}\left[h(\theta)\frac{p(\theta)}{q(\theta)}\right] &= \mathbb{E}_{\theta\sim q}\left[h(\theta)^2\frac{p(\theta)^2}{q(\theta)^2}\right] - \left(\mathbb{E}_{\theta\sim q}\left[h(\theta)\frac{p(\theta)}{q(\theta)}\right]\right)^2 \\ &= \mathbb{E}_{\theta\sim q}\left[h(\theta)^2\frac{p(\theta)^2}{q(\theta)^2}\right] - \left(\mathbb{E}_{\theta\sim p}[h(\theta)]\right)^2\end{aligned}$$

and observe that the second term does not depend on q!

First, we probe a general lower bound for the first term. Using Jensen's inequality, we know that

$$\mathbb{E}_{\theta\sim q}\left[h(\theta)^2\frac{p(\theta)^2}{q(\theta)^2}\right] \geq \left(\mathbb{E}_{\theta\sim q}\left[|h(\theta)|\frac{p(\theta)}{q(\theta)}\right]\right)^2 = \left(\mathbb{E}_{\theta\sim p}\left[|h(\theta)|\right]\right)^2$$

Now, let us show that the lower bound is achieved when $q = q^*$

$$\mathbb{E}_{\theta\sim q^*}\left[h(\theta)^2\frac{p(\theta)^2}{q^*(\theta)^2}\right] = \mathbb{E}\left[h(\theta)^2\frac{p(\theta)^2}{|h(\theta)|^2p(\theta)^2}\int |h(z)|p(z)\mathrm{d}z\right] = \left(\mathbb{E}_{\theta\sim p}\left[|h(\theta)|\right]\right)^2$$

Rejection Sampling

In the following, we use rejection sampling to sample from the full conditional of σ in our model for the analysis of the `VetBP` dataset.

```
# Target unnormalized log-posterior/density for sigma
log_f <- function(sigma, ndata, sumsq) {
  if (sigma <= 0) return(-Inf)
  - (ndata + 1) * log(sigma) - 0.5 / (sigma^2) + sumsq -
    log(sigma * 0.5) - 0.5 * (log(sigma) - log(13.43))^2
}

# Proposal log-density (Inverse-Gamma)
log_g <- function(sigma, ndata, sumsq) {
  - (ndata + 1) * log(sigma) - 0.5 / (sigma^2) + sumsq
}

# Rejection sampling for full conditional of sigma
sample_fullcond_sigma <- function(data, mu) {
  n <- length(data)
  sumsq <- sum((data - mu)^2)

  shape_param <- (n - 1) / 2.0
  rate_param  <- sumsq / 2.0

  repeat {
```

```
    # Draw from Inverse-Gamma(shape, rate) by inverting a Gamma draw
    gamma_draw <- rgamma(1, shape = shape_param, rate = rate_param)
    prop <- sqrt(1 / gamma_draw)

    # Draw uniform acceptance threshold
    u <- runif(1)

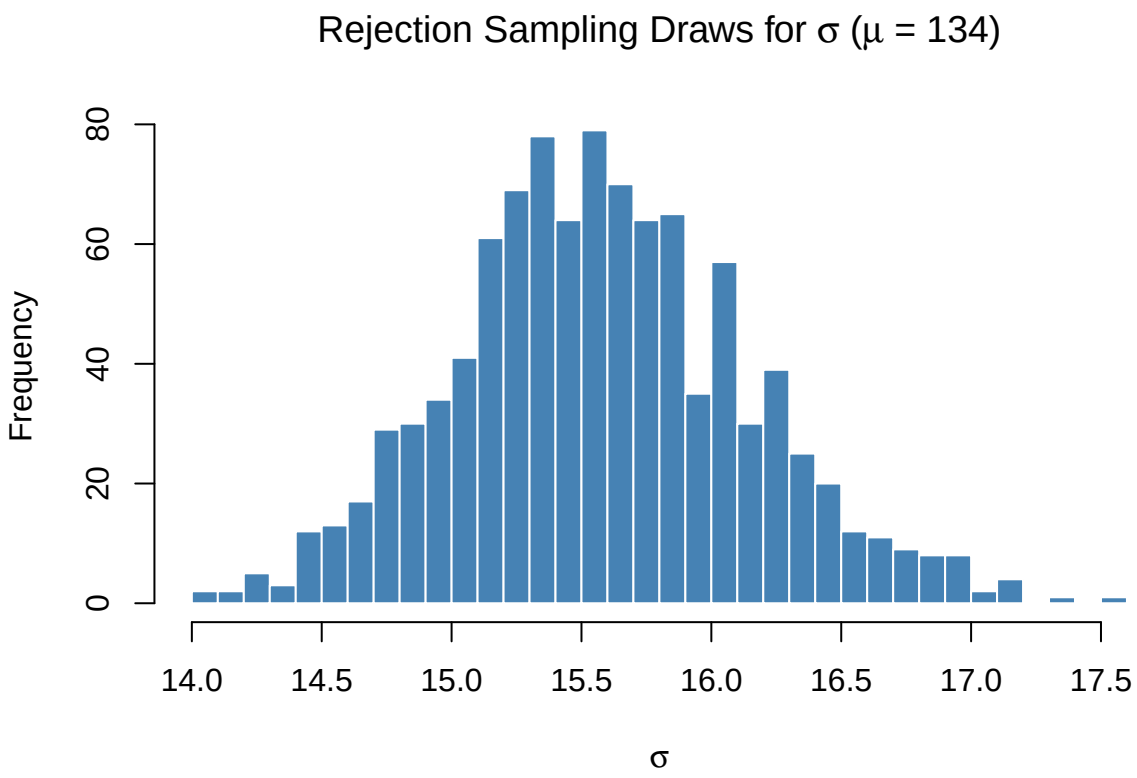
    # Accept/Reject decision in log-space
    log_alpha <- log_f(prop, n, sumsq) - log_g(prop, n, sumsq)

    if (log(u) < log_alpha) {
      return(prop)
    }
  }
}
```

```
set.seed(42)

# Condition on mu = 134 and draw 1000 samples
rej_samps <- replicate(1000, sample_fullcond_sigma(sbp, 134))

hist(
  rej_samps,
  breaks = 30,
  col = "steelblue",
  border = "white",
  xlab = expression(sigma),
  main = expression("Rejection Sampling Draws for " * sigma * " (" * mu * " = 134)")
)
```



Homework

Try to sample from the normal distribution $\mathcal{N}(0, 1)$ truncated on [a, b] using rejection sampling. As proposal, you can use $\mathcal{N}(0, 1)$.

What happens when

1. [a, b] is centered around 0 and large (e.g. [-1, 1])
2. [a, b] is centered around 0 and small (e.g. [-0.5, 0.5])
3. [a, b] moves to one of the tail (e.g. consider $[a_n, b_n]$ = [-1 + n, 1+n])